9

Using DDL Statements to Create and Manage Tables

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Objectives

After completing this lesson, you should be able to do the following:

- **Categorize the main database objects**
- **Review the table structure**
- **List the data types that are available for columns**
- **Create a simple table**
- **Explain how constraints are created at the time of table creation**
- **Describe how schema objects work**

ORACLE

9 - 2 Copyright © 2006, Oracle. All rights reserved.

Objectives

In this lesson, you are introduced to the data definition language (DDL) statements. You are taught the basics of how to create simple tables, alter them, and remove them. The data types available in DDL are shown, and schema concepts are introduced. Constraints are tied into this lesson. Exception messages that are generated from violating constraints during DML are shown and explained.

Database Objects

Object	Description
Table	Basic unit of storage; composed of rows
View	Logically represents subsets of data from one or more tables
Sequence	Generates numeric values
Index	Improves the performance of some queries
Synonym	Gives alternative names to objects

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Database Objects

An Oracle database can contain multiple data structures. Each structure should be outlined in the database design so that it can be created during the build stage of database development.

- **Table:** Stores data
- **View:** Subset of data from one or more tables
- **Sequence:** Generates numeric values
- **Index:** Improves the performance of some queries
- **Synonym:** Gives alternative names to objects

Oracle Table Structures

- Tables can be created at any time, even while users are using the database.
- You do not need to specify the size of a table. The size is ultimately defined by the amount of space allocated to the database as a whole. It is important, however, to estimate how much space a table will use over time.
- Table structure can be modified online.

Note: More database objects are available but are not covered in this course.

Naming Rules

Table names and column names:

- **Must begin with a letter**
- **Must be 1–30 characters long**
- **Must contain only A–Z, a–z, 0–9, _, $, and #**
- **Must not duplicate the name of another object owned by the same user**
- **Must not be an Oracle server–reserved word**

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Naming Rules

You name database tables and columns according to the standard rules for naming any Oracle database object:

- Table names and column names must begin with a letter and be 1–30 characters long.
- Names must contain only the characters A–Z, a–z, 0–9, _ (underscore), $, and # (legal characters, but their use is discouraged).
- Names must not duplicate the name of another object owned by the same Oracle server user.
- Names must not be an Oracle server–reserved word.

Naming Guidelines

Use descriptive names for tables and other database objects.

Note: Names are case-insensitive. For example, `EMPLOYEES` is treated as the same name as `eMPloyees` or `eMpLOYEES`.

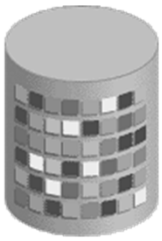
For more information, see “Object Names and Qualifiers” in the *Oracle Database SQL Reference.*

CREATE TABLE Statement

- **You must have:**
  - **CREATE TABLE privilege**
  - **A storage area**

```
CREATE TABLE [schema.]table
          (column datatype [DEFAULT expr][, ...]);
```

- **You specify:**
  - **Table name**
  - **Column name, column data type, and column size**



ORACLE

Copyright © 2006, Oracle. All rights reserved.

CREATE TABLE Statement

You create tables to store data by executing the SQL `CREATE TABLE` statement. This statement is one of the DDL statements, which are a subset of SQL statements used to create, modify, or remove Oracle database structures. These statements have an immediate effect on the database, and they also record information in the data dictionary.

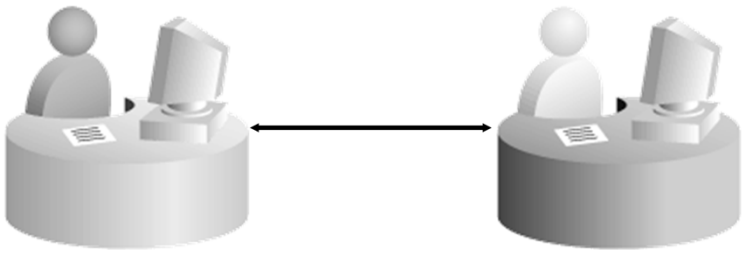
To create a table, a user must have the `CREATE TABLE` privilege and a storage area in which to create objects. The database administrator uses data control language statements to grant privileges to users (DCL statements are covered in a later lesson).

In the syntax:

schema	Is the same as the owner's name
table	is the name of the table
DEFAULT *expr*	Specifies a default value if a value is omitted in the `INSERT` statement
column	Is the name of the column
datatype	Is the column's data type and length

Referencing Another User's Tables

- **Tables belonging to other users are not in the user's schema.**
- **You should use the owner's name as a prefix to those tables.**



USERA

```
SELECT *
FROM userB.employees;
```

USERB

```
SELECT *
FROM userA.employees;
```


Copyright © 2006, Oracle. All rights reserved.

Referencing Another User's Tables

A *schema* is a collection of objects. Schema objects are the logical structures that directly refer to the data in a database. Schema objects include tables, views, synonyms, sequences, stored procedures, indexes, clusters, and database links.

If a table does not belong to the user, the owner's name must be prefixed to the table. For example, if there are schemas named `USERA` and `USERB`, and both have an `EMPLOYEES` table, then if `USERA` wants to access the `EMPLOYEES` table that belongs to `USERB`, he must prefix the table name with the schema name:

```
SELECT *
FROM   userb.employees;
```

If `USERB` wants to access the `EMPLOYEES` table that is owned by `USERA`, he must prefix the table name with the schema name:

```
SELECT *
FROM   usera.employees;
```

DEFAULT Option

- Specify a default value for a column during an insert.

```
... hire_date DATE DEFAULT SYSDATE, ...
```

- Literal values, expressions, or SQL functions are legal values.
- Another column's name or a pseudocolumn are illegal values.
- The default data type must match the column data type.

```
CREATE TABLE hire_dates
        (id          NUMBER(8),
        hire_date DATE DEFAULT SYSDATE);
Table created.
```

ORACLE

9 - 7 Copyright © 2006, Oracle. All rights reserved.

DEFAULT Option

When you define a table, you can specify that a column be given a default value by using the DEFAULT option. This option prevents null values from entering the columns if a row is inserted without a value for the column. The default value can be a literal, an expression, or a SQL function (such as SYSDATE or USER), but the value cannot be the name of another column or a pseudocolumn (such as NEXTVAL or CURRVAL). The default expression must match the data type of the column.

Note: CURRVAL and NEXTVAL are explained later in this lesson.

Creating Tables

- **Create the table.**

```
CREATE TABLE dept
        (deptno      NUMBER(2),
         dname       VARCHAR2(14),
         loc         VARCHAR2(13),
         create_date DATE DEFAULT SYSDATE);
Table created.
```

- **Confirm table creation.**

```
DESCRIBE dept
```

Name	Null?	Type
DEPTNO		NUMBER(2)
DNAME		VARCHAR2(14)
LOC		VARCHAR2(13)
CREATE_DATE		DATE

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Creating Tables

The example in the slide creates the DEPT table, with four columns: DEPTNO, DNAME, LOC, and CREATE_DATE. The CREATE_DATE column has a default value. If a value is not provided for an INSERT statement, the system date is automatically inserted.

It further confirms the creation of the table by issuing the DESCRIBE command.

Because creating a table is a DDL statement, an automatic commit takes place when this statement is executed.

Data Types

Data Type	Description
VARCHAR2(*size*)	Variable-length character data
CHAR(*size*)	Fixed-length character data
NUMBER(*p*,*s*)	Variable-length numeric data
DATE	Date and time values
LONG	Variable-length character data (up to 2 GB)
CLOB	Character data (up to 4 GB)
RAW and LONG RAW	Raw binary data
BLOB	Binary data (up to 4 GB)
BFILE	Binary data stored in an external file (up to 4 GB)
ROWID	A base-64 number system representing the unique address of a row in its table

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Data Types

When you identify a column for a table, you need to provide a data type for the column. There are several data types available:

Data Type	Description
VARCHAR2(*size*)	Variable-length character data (A maximum *size* must be specified: minimum *size* is 1; maximum *size* is 4,000.)
CHAR [(*size*)]	Fixed-length character data of length *size* bytes (Default and minimum *size* is 1; maximum *size* is 2,000.)
NUMBER [(*p*,*s*)]	Number having precision *p* and scale *s* (The precision is the total number of decimal digits, and the scale is the number of digits to the right of the decimal point; the precision can range from 1 to 38, and the scale can range from –84 to 127.)
DATE	Date and time values to the nearest second between January 1, 4712 B.C., and December 31, 9999 A.D.
LONG	Variable-length character data (up to 2 GB)
CLOB	Character data (up to 4 GB)

Data Types (continued)

Data Type	Description
RAW(*size*)	Raw binary data of length *size* (A maximum *size* must be specified: maximum *size* is 2,000.)
LONG RAW	Raw binary data of variable length (up to 2 GB)
BLOB	Binary data (up to 4 GB)
BFILE	Binary data stored in an external file (up to 4 GB)
ROWID	A base-64 number system representing the unique address of a row in its table

Guidelines

- A LONG column is not copied when a table is created using a subquery.
- A LONG column cannot be included in a GROUP BY or an ORDER BY clause.
- Only one LONG column can be used per table.
- No constraints can be defined on a LONG column.
- You might want to use a CLOB column rather than a LONG column.

Datetime Data Types

You can use several datetime data types:

Data Type	Description
TIMESTAMP	Date with fractional seconds
INTERVAL YEAR TO MONTH	Stored as an interval of years and months
INTERVAL DAY TO SECOND	Stored as an interval of days, hours, minutes, and seconds

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Other Datetime Data Types

Data Type	Description
TIMESTAMP	Enables the time to be stored as a date with fractional seconds. There are several variations of this data type.
INTERVAL YEAR TO MONTH	Enables time to be stored as an interval of years and months. Used to represent the difference between two datetime values in which the only significant portions are the year and month.
INTERVAL DAY TO SECOND	Enables time to be stored as an interval of days, hours, minutes, and seconds. Used to represent the precise difference between two datetime values.

Note: These datetime data types are available with Oracle9*i* and later releases. For detailed information about the datetime data types, see the topics "TIMESTAMP Datatype," "INTERVAL YEAR TO MONTH Datatype," and "INTERVAL DAY TO SECOND Datatype" in the *Oracle SQL Reference*.

Datetime Data Types

- The **TIMESTAMP** data type is an extension of the **DATE** data type.
- It stores the year, month, and day of the **DATE** data type plus hour, minute, and second values as well as the fractional second value.
- You can optionally specify the time zone.

```
TIMESTAMP[(fractional_seconds_precision)]
```

```
TIMESTAMP[(fractional_seconds_precision)]
WITH TIME ZONE
```

```
TIMESTAMP[(fractional_seconds_precision)]
WITH LOCAL TIME ZONE
```

ORACLE

Copyright © 2006, Oracle. All rights reserved.

TIMESTAMP Data Type

The TIMESTAMP data type is an extension of the DATE data type. It stores the year, month, and day of the DATE data type plus hour, minute, and second values. This data type is used for storing precise time values.

The fractional_seconds_precision optionally specifies the number of digits in the fractional part of the SECOND datetime field and can be a number in the range 0 to 9. The default is 6.

Example

In this example, a table is created named NEW_EMPLOYEES, with a column START_DATE that has a data type of TIMESTAMP:

```
CREATE TABLE new_employees
  (employee_id NUMBER,
   first_name VARCHAR2(15),
   last_name VARCHAR2(15),
   ...
   start_date TIMESTAMP(7),
   ...);
```

Suppose that two rows are inserted in the NEW_EMPLOYEES table. The displayed output shows the differences. (A DATE data type defaults to display the DD-MON-RR format.):

TIMESTAMP Data Type (continued)

```
SELECT start_date
FROM   new_employees;

17-JUN-03 12.00.00.000000 AM
21-SEP-03 12.00.00.000000 AM
```

TIMESTAMP WITH TIME ZONE Data Type

TIMESTAMP WITH TIME ZONE is a variant of TIMESTAMP that includes a time-zone displacement in its value. The time-zone displacement is the difference (in hours and minutes) between local time and UTC (Universal Time Coordinate, formerly known as Greenwich Mean Time). This data type is used for collecting and evaluating date information across geographic regions.

For example,

```
TIMESTAMP '2003-04-15 8:00:00 -8:00'
```

is the same as

```
TIMESTAMP '2003-04-15 11:00:00 -5:00'
```

That is, 8:00 a.m. Pacific Standard Time is the same as 11:00 a.m. Eastern Standard Time.

This can also be specified as follows:

```
TIMESTAMP '2003-04-15 8:00:00 US/Pacific'
```

TIMESTAMP WITH LOCAL TIME ZONE Data Type

TIMESTAMP WITH LOCAL TIME ZONE is another variant of TIMESTAMP that includes a time-zone displacement in its value. It differs from TIMESTAMP WITH TIME ZONE in that data stored in the database is normalized to the database time zone, and the time-zone displacement is not stored as part of the column data. When users retrieve the data, it is returned in the users' local session time zone. The time-zone displacement is the difference (in hours and minutes) between local time and UTC.

Unlike TIMESTAMP WITH TIME ZONE, you can specify columns of type TIMESTAMP WITH LOCAL TIME ZONE as part of a primary or unique key, as in the following example:

```
CREATE TABLE time_example
    (order_date TIMESTAMP WITH LOCAL TIME ZONE);

INSERT INTO time_example VALUES('15-JAN-04 09:34:28 AM');

SELECT *
FROM   time_example;

ORDER_DATE
----------------------------
15-JAN-04 09.34.28.000000 AM
```

The TIMESTAMP WITH LOCAL TIME ZONE type is appropriate for two-tier applications in which you want to display dates and times using the time zone of the client system.

Datetime Data Types

- The `INTERVAL YEAR TO MONTH` data type stores a period of time using the `YEAR` and `MONTH` datetime fields:

```
INTERVAL YEAR [(year_precision)] TO MONTH
```

- The `INTERVAL DAY TO SECOND` data type stores a period of time in terms of days, hours, minutes, and seconds:

```
INTERVAL DAY [(day_precision)]
   TO SECOND [(fractional_seconds_precision)]
```

ORACLE

Copyright © 2006, Oracle. All rights reserved.

`INTERVAL YEAR TO MONTH` Data Type

`INTERVAL YEAR TO MONTH` stores a period of time using the `YEAR` and `MONTH` datetime fields.

Use `INTERVAL YEAR TO MONTH` to represent the difference between two datetime values, where the only significant portions are the year and month. For example, you might use this value to set a reminder for a date that is 120 months in the future, or check whether 6 months have elapsed since a particular date.

In the syntax:

`year_precision`	is the number of digits in the `YEAR` datetime field. The default value of `year_precision` is 2.

Examples

- `INTERVAL '123-2' YEAR(3) TO MONTH`
  Indicates an interval of 123 years, 2 months
- `INTERVAL '123' YEAR(3)`
  Indicates an interval of 123 years 0 months
- `INTERVAL '300' MONTH(3)`
  Indicates an interval of 300 months
- `INTERVAL '123' YEAR`
  Returns an error because the default precision is 2, and `123` has 3 digits

INTERVAL YEAR TO MONTH Data Type (continued)

```
CREATE TABLE time_example2
(loan_duration INTERVAL YEAR (3) TO MONTH);

INSERT INTO time_example2 (loan_duration)
  VALUES (INTERVAL '120' MONTH(3));

SELECT TO_CHAR( sysdate+loan_duration, 'dd-mon-yyyy')
FROM   time_example2;        --today's date is 26-Sep-2001
```

TO_CHAR(SYS
26-sep-2011

INTERVAL DAY TO SECOND Data Type

`INTERVAL DAY TO SECOND` stores a period of time in terms of days, hours, minutes, and seconds.

Use `INTERVAL DAY TO SECOND` to represent the precise difference between two datetime values. For example, you might use this value to set a reminder for a time that is 36 hours in the future, or to record the time between the start and end of a race. To represent long spans of time, including multiple years, with high precision, you can use a large value for the days portion.

In the syntax:

`day_precision`	Is the number of digits in the `DAY` datetime field. Accepted values are 0 to 9. The default is 2.
`fractional_seconds_precision`	Is the number of digits in the fractional part of the `SECOND` datetime field. Accepted values are 0 to 9. The default is 6.

Examples

- `INTERVAL '4 5:12:10.222' DAY TO SECOND(3)`
  Indicates 4 days, 5 hours, 12 minutes, 10 seconds, and 222 thousandths of a second.
- `INTERVAL '180' DAY(3)`
  Indicates 180 days.
- `INTERVAL '4 5:12:10.222' DAY TO SECOND(3)`
  Indicates 4 days, 5 hours, 12 minutes, 10 seconds, and 222 thousandths of a second
- `INTERVAL '4 5:12' DAY TO MINUTE`
  Indicates 4 days, 5 hours, and 12 minutes
- `INTERVAL '400 5' DAY(3) TO HOUR`
  Indicates 400 days and 5 hours.
- `INTERVAL '11:12:10.2222222' HOUR TO SECOND(7)`
  Indicates 11 hours, 12 minutes, and 10.2222222 seconds.

INTERVAL DAY TO SECOND Data Type (continued)

Example

```
CREATE TABLE time_example3
(day_duration INTERVAL DAY (3) TO SECOND);

INSERT INTO time_example3 (day_duration)
VALUES (INTERVAL '180' DAY(3));

SELECT sysdate + day_duration "Half Year"
FROM   time_example3;        --today's date is 26-Sep-2001
```

Half Year
25-MAR-02